

Task 1:

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output and Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

A) To split into tokens

- ☐ Break it into meaningful units called tokens.

Example:

`ls -l /home`

➔ Tokens:

`ls`
`-l`
`/home`

```
varshini@DESKTOP-BLT5L3R:~$ mkdir task1
cd task1
varshini@DESKTOP-BLT5L3R:~/task1$ nano parser.c
varshini@DESKTOP-BLT5L3R:~/task1$ gcc parser.c -o parser
varshini@DESKTOP-BLT5L3R:~/task1$ ./parser
Enter command: ls -l /home

Tokens:
Token 1: ls
Token 2: -l
Token 3: /home
```

A **delimiter** is a character that separates one part of the command from another.

Whitespace means spaces, tabs, and newline characters.

There may be multiple spaces between tokens.

The parser normally treats them as separators rather than meaningful data.

Handling whitespace prevents unnecessary spaces from becoming tokens.

```
GNU nano 7.2 parser.c
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 1000
#define MAX_TOKENS 100

int main() {
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    // Remove newline
    input[strcspn(input, "\n")] = '\0';

    // Check empty command
    if (strlen(input) == 0) {
        printf("Empty command\n");
        return 0;
    }

    // Split input using spaces and tabs
    char *token = strtok(input, " \t");

    while (token != NULL && count < MAX_TOKENS) {
        tokens[count++] = token;
        token = strtok(NULL, " \t");
    }

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++) {
        printf("Token %d: %s\n", i + 1, tokens[i]);
    }

    return 0;
}

[ Read 39 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Instead of storing tokens as simple strings, we can create a **token structure** containing information about each token.

Example:

ls -l | grep txt

can become:

Token 1 → ls → COMMAND
Token 2 → -l → ARGUMENT
Token 3 → | → PIPE
Token 4 → grep → COMMAND
Token 5 → txt → ARGUMENT

Tokenizers:

```
varshini@DESKTOP-BLT5L3R:~/task1$ nano tokenizer.c
varshini@DESKTOP-BLT5L3R:~/task1$ ^C
varshini@DESKTOP-BLT5L3R:~/task1$ gcc tokenizer.c -o tokenizer
varshini@DESKTOP-BLT5L3R:~/task1$ ./tokenizer
Enter command: ls -l | grep txt
```

Tokens:

```
WORD      : ls
WORD      : -l
DELIMITER : |
WORD      : grep
WORD      : txt
```

```
GNU nano 7.2 tokenizer.c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1000
#define MAX_TOKEN 100

int main() {
    char input[MAX_INPUT];
    char token[MAX_TOKEN];
    int i = 0, j = 0;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    printf("\nTokens:\n");

    while (input[i] != '\0' && input[i] != '\n') {
        // Ignore whitespace
        if (isspace(input[i])) {
            if (j > 0) {
                token[j] = '\0';
                printf("WORD      : %s\n", token);
                j = 0;
            }
            i++;
        }

        // Handle special delimiters
        else if (input[i] == '|' ||
                input[i] == '>' ||
                input[i] == '<' ||
                input[i] == ';') {
            if (j > 0) {
                token[j] = '\0';
                printf("WORD      : %s\n", token);
                j = 0;
            }

            printf("DELIMITER : %c\n", input[i]);
            i++;
        }

        // Normal character
        else {
            token[j++] = input[i++];
        }
    }

    // Print final token
    if (j > 0) {
        token[j] = '\0';
        printf("WORD      : %s\n", token);
    }

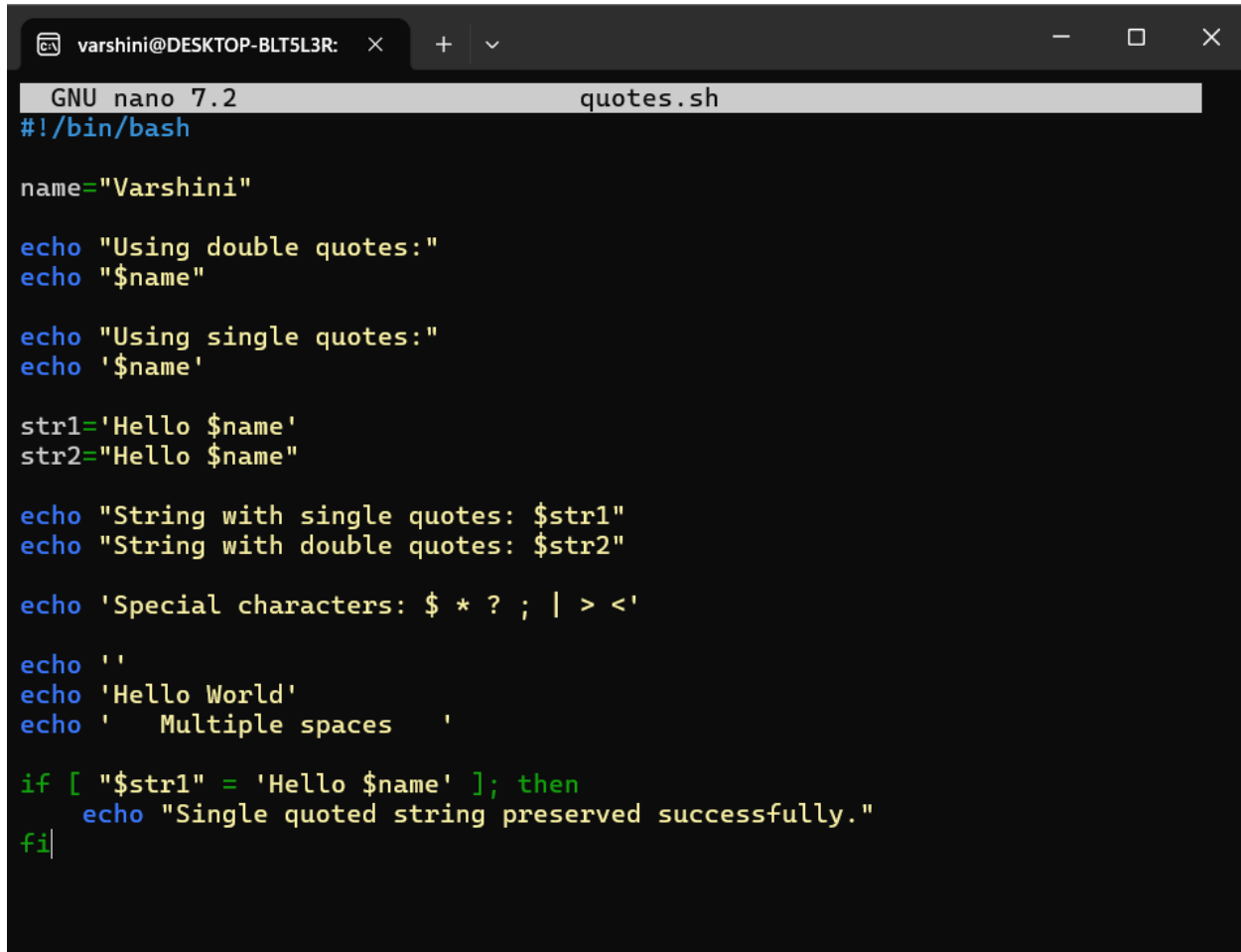
    return 0;
}
```

[Read 59 lines]

^G Help	^O Write Out	^W Where Is	^K Cut	^T Execute	^C Location	^M-U Undo
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/_ Go To Line	^M-E Redo

Task 2:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.



```
varshini@DESKTOP-BLT5L3R: × + ▾
GNU nano 7.2 quotes.sh
#!/bin/bash

name="Varshini"

echo "Using double quotes:"
echo "$name"

echo "Using single quotes:"
echo '$name'

str1='Hello $name'
str2="Hello $name"

echo "String with single quotes: $str1"
echo "String with double quotes: $str2"

echo 'Special characters: $ * ? ; | > <'

echo ''
echo 'Hello World'
echo '  Multiple spaces  '

if [ "$str1" = 'Hello $name' ]; then
    echo "Single quoted string preserved successfully."
fi
```

```
varshini@DESKTOP-BLT5L3R:~$ nano quotes.sh
varshini@DESKTOP-BLT5L3R:~$ LS
LS: command not found
varshini@DESKTOP-BLT5L3R:~$ ls
OSSP      fork_exec  hello.c    process.c  shell
file1.txt  fork_exec.c  main.c     quotes.sh  shortcut
file2.txt  hello       myfolder  script.sh  task1
varshini@DESKTOP-BLT5L3R:~$ chmod +x quotes.sh
varshini@DESKTOP-BLT5L3R:~$ ./quotes.sh
Using double quotes:
Varshini
Using single quotes:
$name
String with single quotes: Hello $name
String with double quotes: Hello Varshini
Special characters: $ * ? ; | > <

Hello World
    Multiple spaces
Single quoted string preserved successfully.
varshini@DESKTOP-BLT5L3R:~$ bash quotes.sh
Using double quotes:
Varshini
Using single quotes:
$name
String with single quotes: Hello $name
String with double quotes: Hello Varshini
Special characters: $ * ? ; | > <

Hello World
    Multiple spaces
Single quoted string preserved successfully.
```

```
GNU nano 7.2          quotes.sh
#!/bin/bash

name="Varshini"

echo "Using double quotes:"
echo "$name"

echo "Using single quotes:"
echo '$name'

str1='Hello $name'
str2="Hello $name"

echo "String with single quotes: $str1"
echo "String with double quotes: $str2"

echo 'Special characters: $ * ? ; | > < '

echo ''
echo 'Hello World'
echo '  Multiple spaces  '

if [ "$str1" = 'Hello $name' ]; then
    echo "Single quoted string preserved successfully."
fi
```

Observation

1. Single quotes preserve the string literally and prevent variable expansion.
2. Double quotes allow variable expansion using \$variable.
3. Special characters inside single quotes are treated as literal characters.
4. The program successfully validates quoted strings and handles edge cases such as empty strings and spaces.

Task 3:

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
varshini@DESKTOP-BLT5L3R:~$ cd ~
varshini@DESKTOP-BLT5L3R:~$ pwd
/home/varshini
varshini@DESKTOP-BLT5L3R:~$ nano myshell.c
varshini@DESKTOP-BLT5L3R:~$ nano myshell.c
varshini@DESKTOP-BLT5L3R:~$ gcc myshell.c -o myshell
varshini@DESKTOP-BLT5L3R:~$ ls\
```

```
>
OSSP      fork_exec  hello.c  myshell  quotes.sh  shortcut
file1.txt fork_exec.c  main.c  myshell.c  script.sh  task1
file2.txt hello      myfolder process.c  shell
```

```
GNU nano 7.2 myshell.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 100
#define MAX_INPUT 1024

int parse_command(char *input, char *args[]) {
    int count = 0;
    char *p = input;

    while (*p) {
        while (*p == ' ' || *p == '\t')
            p++;

        if (*p == '\0' || *p == '\n')
            break;

        if (*p == '"') {
            p++;
            char *start = p;

            while (*p != '"' && *p != '\0' && *p != '\n')
                p++;

            if (*p != '"') {
                printf("Syntax Error: Unmatched double quote.\n");
                return -1;
            }

            *p = '\0';

            char expanded[MAX_INPUT];
            char *user_pos = strstr(start, "$USER");

            if (user_pos) {
                char *username = getenv("USER");

                int prefix_len = user_pos - start;

                snprintf(expanded, sizeof(expanded),
                        "%.s%s%s",
                        prefix_len,
                        start,
                        username ? username : "",
                        user_pos + 5);

                args[count++] = strdup(expanded);
            } else {
                args[count++] = strdup(start);
            }

            p++;
        } else {
            char *start = p;

            while (*p != ' ' && *p != '\t' &&
                    *p != '\n' && *p != '\0')
                p++;

            if (*p != '\0') {
                *p = '\0';
                p++;
            }

            args[count++] = strdup(start);
        }
    }

    args[count] = NULL;
    return count;
}
```

```

int main() {
    char input[MAX_INPUT];

    while (1) {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (strcmp(input, "exit\n") == 0)
            break;

        char *args[MAX_ARGS];

        int argc = parse_command(input, args);

        if (argc == -1)
            continue;

        if (argc == 0)
            continue;

        printf("\n--- Parsed Arguments ---\n\n");

        for (int i = 0; i < argc; i++)
            printf("Argument %d: %s\n", i + 1, args[i]);

        pid_t pid = fork();

        if (pid == 0) {
            printf("\nChild Process\n");
            printf("PID = %d\n\n", getpid());

            execvp(args[0], args);

            perror("Command execution failed");
            exit(1);
        }

        else if (pid > 0) {
            wait(NULL);
            printf("\nChild process completed.\n");
        }

        else {
            perror("fork failed");
        }

        for (int i = 0; i < argc; i++)
            free(args[i]);
    }

    return 0;
}

```

Test Case 1

```
varshini@DESKTOP-BLT5L3R:~$ ./myshell
myShell> echo "Hello $USER"

--- Parsed Arguments ---

Argument 1: echo
Argument 2: Hello varshini

Child Process
PID = 619

Hello varshini

Child process completed.
myShell>
```

Test Case 2

```
myShell> echo "Hello World
Syntax Error: Unmatched double quote.
myShell> exit|
```

Observation

1. Double quotes preserve spaces and allow variable expansion such as \$USER.
2. The shell successfully parses quoted strings as single arguments.
3. A child process is created using fork() and commands are executed using execvp().
4. The program displays the child PID and waits for the child process to complete.
5. Unmatched double quotes are detected and reported as a syntax error.

Task 4: To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```
GNU nano 7.2                                escape.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 100
#define MAX_INPUT 1024

int parse_command(char *input, char *args[]) {
    int count = 0;
    char token[MAX_INPUT];
    int j = 0;
    int escaped = 0;

    for (int i = 0; input[i] != '\0' && input[i] != '\n'; i++) {
        if (escaped) {
            token[j++] = input[i];
            escaped = 0;
        }

        else if (input[i] == '\\') {
            escaped = 1;
        }

        else if (input[i] == ' ' || input[i] == '\t') {
            if (j > 0) {
                token[j] = '\0';
                args[count++] = strdup(token);
                j = 0;
            }
        }

        else {
            token[j++] = input[i];
        }
    }

    if (escaped) {
        token[j++] = '\\';
    }

    if (j > 0) {
        token[j] = '\0';
        args[count++] = strdup(token);
    }

    args[count] = NULL;
    return count;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```

int main() {
    char input[MAX_INPUT];

    while (1) {
        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (strcmp(input, "exit\n") == 0)
            break;

        char *args[MAX_ARGS];

        int argc = parse_command(input, args);

        if (argc == 0)
            continue;

        printf("\n--- Parsed Arguments ---\n\n");

        for (int i = 0; i < argc; i++)
            printf("Argument %d: %s\n", i + 1, args[i]);

        pid_t pid = fork();

        if (pid == 0) {
            printf("\nChild Process\n");
            printf("PID = %d\n\n", getpid());

            execvp(args[0], args);

            perror("Command execution failed");
            exit(1);
        }

        else if (pid > 0) {
            wait(NULL);
            printf("\nChild process completed.\n");
        }

        else {
            perror("fork failed");
        }

        for (int i = 0; i < argc; i++)
            free(args[i]);
    }

    return 0;
}

```

```
myShell> echo Hello\ World

--- Parsed Arguments ---

Argument 1: echo
Argument 2: Hello World

Child Process
PID = 698

Hello World

Child process completed.
myShell> echo Hello\&World

--- Parsed Arguments ---

Argument 1: echo
Argument 2: Hello&World

Child Process
PID = 735

Hello&World

Child process completed.
myShell> echo This\ is\ a\ test

--- Parsed Arguments ---

Argument 1: echo
Argument 2: This is a test

Child Process
PID = 736

This is a test

Child process completed.
myShell> |
```

Observation

1. Escape character \ allows **spaces and special symbols to be treated as literal characters.**
2. Escaped spaces are preserved as part of a **single argument.**
3. The parser correctly handles **escaped characters and complex inputs.**
4. Parsed arguments are validated, and commands are executed using a **child process.**

Task 5: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
GNU nano 7.2 process.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char *args[] = {"echo", "Hello from Child Process", NULL};

    printf("Parent Process\n");
    printf("Parent PID = %d\n", getpid());

    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }

    else if (pid == 0) {
        printf("\nChild Process\n");
        printf("Child PID = %d\n", getpid());

        printf("Executing command: echo Hello from Child Process\n");

        execvp(args[0], args);

        perror("Execution failed");
        exit(1);
    }

    else {
        printf("\nParent waiting for child...\n");

        wait(NULL);

        printf("\nChild process completed.\n");
        printf("Parent process continues.\n");
    }

    return 0;
}
```

```
varshini@DESKTOP-BLT5L3R:~$ nano process.c
varshini@DESKTOP-BLT5L3R:~$ gcc process.c -o process
varshini@DESKTOP-BLT5L3R:~$ ./process
Parent Process
Parent PID = 758

Parent waiting for child...

Child Process
Child PID = 759
Executing command: echo Hello from Child Process
Hello from Child Process

Child process completed.
Parent process continues.
```

```

varshini@DESKTOP-BLT5L3R:~$ char *args[] = {"echo", "Hello from Child Process", NULL};
Command 'char' not found, did you mean:
  command 'chr' from snap chr (0.80.4)
  command 'chat' from deb ppp (2.4.9-1+1.1ubuntu1)
  command 'car' from deb ucommon-utils (7.0.1-0.1)
  command 'pchar' from deb pchar (1.5-6)
  command 'shar' from deb sharutils (1:4.15.2-9)
  command 'phar' from deb php-cli (2:8.2+93ubuntu1)
  command 'phar' from deb php8.2-cli (8.2.12-1ubuntu2)
  command 'phar' from deb php8.3-cli (8.3.6-0ubuntu0.24.04.9)
See 'snap info <snapname>' for additional versions.
varshini@DESKTOP-BLT5L3R:~$ char *args[] = {"wrongcommand", NULL};
Command 'char' not found, did you mean:
  command 'chr' from snap chr (0.80.4)
  command 'chat' from deb ppp (2.4.9-1+1.1ubuntu1)
  command 'shar' from deb sharutils (1:4.15.2-9)
  command 'pchar' from deb pchar (1.5-6)
  command 'car' from deb ucommon-utils (7.0.1-0.1)
  command 'phar' from deb php-cli (2:8.2+93ubuntu1)
  command 'phar' from deb php8.2-cli (8.2.12-1ubuntu2)
  command 'phar' from deb php8.3-cli (8.3.6-0ubuntu0.24.04.9)
See 'snap info <snapname>' for additional versions.
varshini@DESKTOP-BLT5L3R:~$ gcc process.c -o process
varshini@DESKTOP-BLT5L3R:~$ ./process
Parent Process
Parent PID = 790

Parent waiting for child...

Child Process
Child PID = 791
Executing command: echo Hello from Child Process
Hello from Child Process

Child process completed.
Parent process continues.
varshini@DESKTOP-BLT5L3R:~$ |

```

Observation

1. `fork()` successfully creates a **child process** from the parent.
2. `execvp()` executes the specified program and passes **arguments** to it.
3. The parent process uses `wait()` to **manage and synchronize** with the child.
4. Invalid commands are detected and **execution errors are handled** using `perror()`.